Software Engineering
Lab 1: Requirements Engineering & UML Use-Case Modelling
42. Internal Microservice Catalog & Health Portal

Name: Shreya C Rao
SRN: PES2UG24CS481
Sem: 5
Sec: H

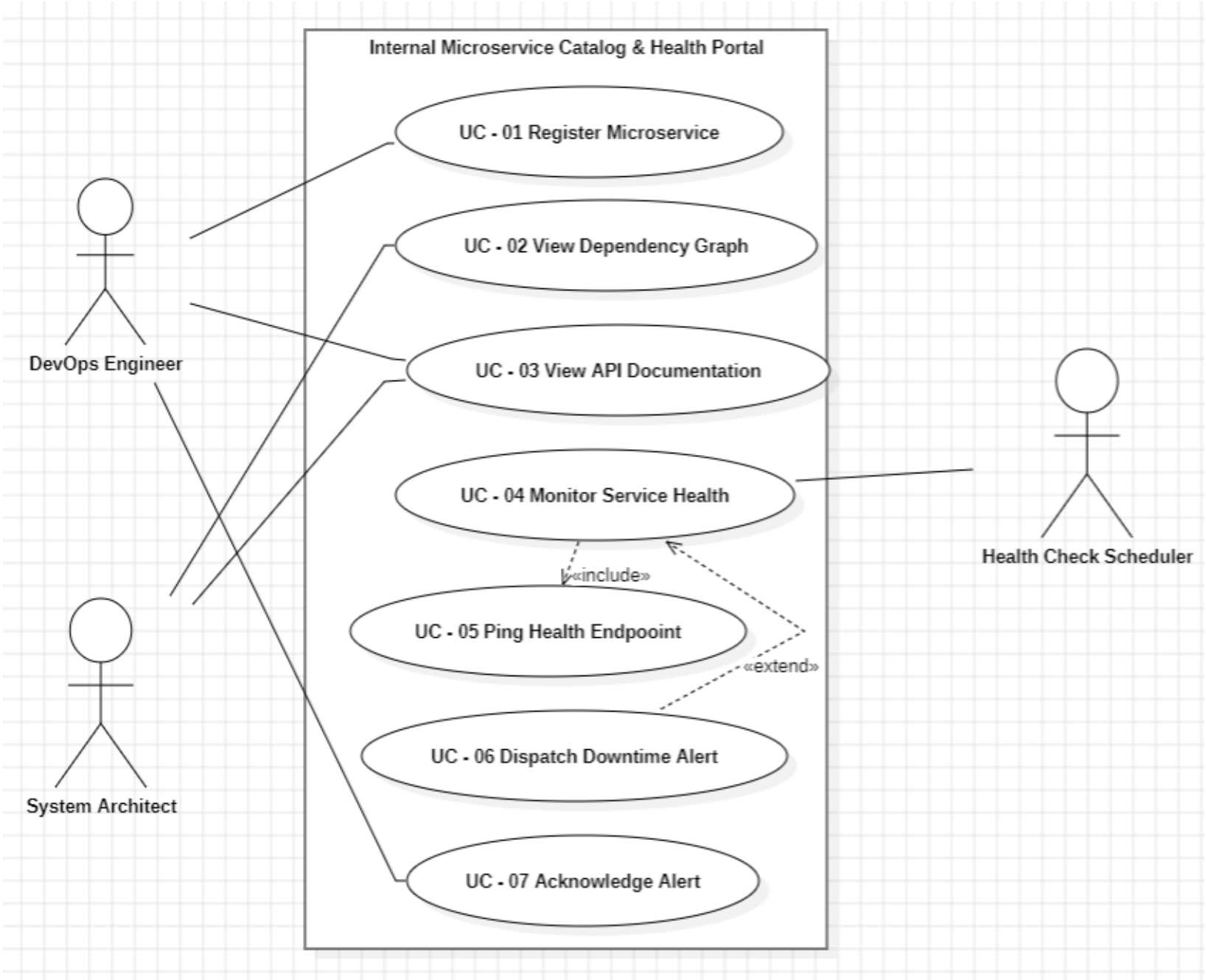
Deliverables :

1. **Complete Requirements Table:**

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-001	Functional	The system shall ping microservice health endpoints at 30-second intervals and compute rolling 24-hour service availability percentages.	High	**Pass:** Downtime is recorded and an alert is dispatched on 3 consecutive failed health probes. **Fail:** An unreachable service is marked healthy.	Continuous health visibility is the core monitoring capability the portal is built around and enables proactive incident response.
FR-002	Functional	The system shall allow a DevOps Engineer to register a new microservice by capturing its name, owning team, endpoint URL, and dependent services.	High	**Pass:** On submitting valid details, the service appears in the catalog with all fields populated and dependency links created within 5 seconds. **Fail:** The service does not appear or dependency links are missing.	Registration is the entry point for the catalog; no other feature has data to operate on without it.
FR-003	Functional	The system shall render an interactive dependency graph showing each registered microservice together with its upstream and downstream dependencies.	Medium	**Pass:** Selecting a service node highlights all of its direct dependencies and dependents. **Fail:** Node selection does not update the connected nodes.	Visualizing dependencies helps System Architects assess blast radius before changes or during an incident.

FR-004	Functional	The system shall aggregate and display the latest API documentation (e.g., an OpenAPI/Swagger spec) for each registered microservice.	Medium	**Pass:** Opening a service's documentation tab shows the current spec version with its endpoint list. **Fail:** The documentation tab is empty or shows a stale spec version.	Centralized API docs reduce the time developers spend hunting for integration details across repositories.
FR-005	Functional	The system shall dispatch a downtime alert to the configured notification channel when a monitored service fails 3 consecutive health probes, and shall allow a DevOps Engineer to acknowledge that alert.	High	**Pass:** An alert is received on the configured channel within 60 seconds of the 3rd consecutive failure, and its status changes to Acknowledged once a DevOps Engineer marks it. **Fail:** The alert is not sent, is duplicated, or acknowledgment does not update its status.	Timely, actionable alerts are what convert passive monitoring data into an operational response.
NFR-001	Non Functional (Performance & Security)	The service catalog dependency graph viewer must render interactive node architectures containing up to 200 services smoothly.	High	**Pass:** Benchmarking tests confirm target latency and security standards under simulated peak load. **Fail:** Rendering drops below an acceptable frame rate, or the viewer exposes unauthorized data under load.	The portal must scale with the organization's growing microservice footprint without degrading usability.
NFR-002	Non Functional (Reliability & Availability)	The health-check polling engine shall maintain at least 99.9% uptime and shall automatically recover any failed pinger job within 60 seconds.	High	**Pass:** 30-day uptime logs show ≥99.9% engine availability and failed jobs restart within 60 seconds. **Fail:** Engine downtime exceeds the threshold, or a failed job is not recovered within 60 seconds.	If the monitoring engine itself is unreliable, the portal's core alerting value proposition fails silently.

2. UML Use-Case Diagram:



3. **Use-Case Flow Specification:**

Use Case: UC – 04 Monitor Service Health
Primary Actor: Health Check Scheduler (automated)
Secondary Actors: DevOps Engineer (receives/acknowledges alerts), Notification Channel
Includes: UC – 05 Ping Health Endpoint (always executed as part of this use case)
Extends: UC – 06 Dispatch Downtime Alert (triggered only when the failure condition is met)

Preconditions

- The microservice being checked is already registered in the catalog with a valid health-check endpoint URL.
- The Health Check Scheduler is running and has an active polling interval configured (30 seconds).

Postconditions

- Success: The service's rolling 24-hour availability percentage is updated and its status (Healthy/Degraded/Down) is current in the catalog.
- Failure: The service is marked Down, a downtime alert has been dispatched, and the incident is visible in the catalog until acknowledged.

Main Success Scenario

1. The Health Check Scheduler triggers a health-check cycle every 30 seconds for each registered microservice.
2. The system invokes UC-05 (Ping Health Endpoint) to send an HTTP request to the microservice's configured health-check endpoint.
3. The microservice responds with a success status within the configured timeout.
4. The system records the probe result with a timestamp and recomputes the rolling 24-hour availability percentage.
5. The system updates the service's status to Healthy in the catalog.
6. The use case ends successfully.

Alternate Flow
3a. Health Probe Fails

- 3a1. The microservice does not respond, times out, or returns an error status.
- 3a2. The system records the failed probe and increments the consecutive-failure counter for that service.

- 3a3. If the consecutive-failure counter reaches 3, the system marks the service as Down and triggers UC-06 (Dispatch Downtime Alert), sending a notification to the configured channel.
- 3a4. The DevOps Engineer receives the alert and may acknowledge it (UC-07), after which the alert status changes to Acknowledged while health checks continue.
- 3a5. Once the service passes a subsequent probe, the system resets the failure counter and updates its status back to Healthy.